Steering Graded SAE Latents Reverses Relational Order in Attention-Only Transformers

Anonymous Authors[1]

Abstract

This document provides a basic paper template and submission guidelines. Abstracts must be a single paragraph, ideally between 4–6 sentences long. Gross violations will trigger corrections at the camera-ready phase. Anonymised code is available here: https://anonymous.4open.science/r/graded-latents-70FA/

1. Introduction

Farrell et al. (2025)

2. Related Work

Mechanistic Interpretability

Mechanistic interpretability is the study of reverse-engineering neural networks into human-interpretable algorithms (Olah et al., 2020; Ferrando et al., 2024; Nanda et al., 2023). This is usually achieved by identifying "circuits" in the model: paths of computation through a model that are responsible for specific behaviours, represented as model subgraphs. Some example behaviours that have been successfully reverse-engineered in this way are indirect object identification (Wang et al., 2023), in-context learning via induction heads (Olsson et al., 2022), and modular arithmetic via Fourier representations (Nanda et al., 2023). Many of these studies use toy models as a controlled environment for identifying mechanisms that might generalise to larger models, which supports our use of toy models in this paper. For example, Baeumel et al. (2025) found the modular arithmetic results on toy models subsequently generalised to pretrained LLMs.

Moreover, recent work has found "steering" vectors in the residual stream which can be steered via linear scaling and patched back into the residual vector to predictably change model behaviour. For example, Arditi et al. (2024) use the difference-in-means vector between residual streams on harmful and harmless instructions to identify a single "refusal direction" in the residual stream of safety-tuned LMs, and use it to to suppress or amplify refusal behaviour via linear scaling. This provides evidence that a given direction in the residual stream is causally relevant for a given behaviour, which provides insights into what feature that direction represents (Ferrando et al., 2024). We use the same principle in our latent steering experiments, in which we steer SAE latents rather than raw residual stream directions and patch the SAE-reconstructed activations into the residual stream.

Relational composition and binding

Classic work in distributed representations show how single vectors can encode structured data. Tensor Product Representations (TRPs) and Vector Symbolic Architectures (VSAs) bind and additively superpose vectors, enabling ordered structures in fixed width vectors (Smolensky, 1990; Plate, 1995; Kanerva, 2009).

Wattenberg & Viégas (2024) highlight additive matrix binding, where vectors are written to slots using role-specific linear maps so that bigrams $(x, y), (y, x)$ remain separable, as a candidate mechanism for relational composition in transformers. That is, given an ordered bigram (d_1, d_2) and two $n \times n$ matrices A and B, define the representation of (x, y) by $r = Ax + By$. In this way, (x, y) and (y, x) have different representations. Wattenberg & Viégas (2024) claim that this causes *feature multiplicity*: a kind of false positive where multiple "echo features" are created that represent the same concept in different contexts. Given two features x and y, matrix binding results in Ax, Ay, Bx, and By features that represent the same concept as x and y.

Empirically, Feng & Steinhardt (2024); Prakash et al. (2024) identify binding ID vectors that bind entities to their attributes in-context in language models. For example, a "green car" is represented by a vector representing "green", a vector for "car", and a binding ID vector to join the two. A binding ID vector k is attached to an entity and its attribute by addition in their respective activation spaces. To answer a query for a given entity, the attribute with a matching binding ID is retrieved from the context activation space. Feng

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Table 1. Comparison of relational composition mechanisms and their implications for interpretability.

Composition	Description	Example	Implications
Direction-based	Features are vectors; sequence position defined by different directions.	Matrix binding (Wattenberg & Viégas, 2024) - features projected to distinct linear subspaces: $r = Ax + By$.	Feature multiplicity: SAEs may learn echo features (Ax, Ay, Bx, By) representing the same underlying concept.
Fixed magnitude-based	Multiple features have same direction; sequence position defined by different magnitudes.	Onion-like representations (Csordás et al., 2024) - "unpeel" via autoregression to retrieve sequence positions.	Counter-example to the widely held assumption that representations are linear.
Relative magnitude-based	Sequence position defined by magnitude relative to other positions in sequence.	This work - sequence order defined by a weighted sum of input token and positional embeddings.	Counter-example to the widely held assumptions that representations are linear and SAE latents are binary.

& Steinhardt (2024) also show that these mechanisms are position-independent with respect to the attribute and entity, and that the activations can be factorised.

Brinkmann et al. (2024) reverse-engineer an attention-only tree-search transformer that stores precomputed subpaths in special token positions and merges them later. This occurs when the tree depth, N, is greater than $L - 1$, where L is the number of transformer layers. This constraint is also necessary in our work, where N is the size of the input N-gram. However, the authors do not investigate the structure of these activations.

Csordás et al. (2024) demonstrate "onion representations" in small gated recurrent neural networks (RNNs), where these slots are defined by different orders of magnitude rather than distinct linear subspaces. The model represents ordered sequences by closing the gates gradually and synchronously over the input phase, which exponentially decays the scaling factor of subsequent sequence positions. This produces layered onion representations where earlier sequence positions dominate the magnitude space. This contrasts larger models which indicate sequence position by sharply closing their gates, which creates position-dependent subspaces for each input. The onion representation allows autoregressive decoding to sequentially "peel" off tokens by subtracting the current dominating token embedding. Multiple tokens can occupy the same directional subspace at different magnitude scales; any linear direction will cross-cut multiple layers of the onion. While the authors find that sequence order is represented by fixed magnitude scales, we find an alternative mechanism where it is instead represented by relative magnitude between positions, which is input-dependent and not fixed.

Our toy model provides evidence for magnitude-based composition in attention-only transformers, but with *relative* magnitudes rather than fixed ones as observed in Csordás et al. (2024). See Table 1 for an overview of these composition methods.

Sparse Autoencoders

Models can store more sparse features than the dimension of their activations naively allows through superposition, creating challenges for interpreting these models (Elhage et al., 2022). Sparse Autoencoders (SAEs) (Bricken et al., 2023a; Cunningham et al., 2024) and their variants (Gao et al., 2025; Bussmann et al., 2024; Rajamanoharan et al., 2024b; Leask et al., 2025b; Costa et al., 2025; Bussmann et al., 2025) have been proposed as a tool for recovering these sparse features from dense activations. An SAE is typically implemented as an autoencoder with a single hidden layer, trained to reconstruct model activations while keeping the hidden representation sparse using a penalty (such as an L_1 regularization term) (Huben et al., 2024; Bricken et al., 2023a). The encoder maps dense activation to a high-dimensional sparse latent space, and the decoder reconstructs the original activations using a learned dictionary of monosemantic latents, which ideally corresponds to interpretable features.

Several architectural variants address limitations in the standard ReLU-based SAE (Table 2). For example, standard L_1 penalties often cause "shrinkage," where the reconstructed feature magnitudes are systematically underestimated (Ferrando et al., 2024). Gated SAEs (Rajamanoharan et al., 2024a) address this by decoupling feature detection from magnitude estimation using a gated mechanism. JumpReLU SAEs (Rajamanoharan et al., 2024b) replace the standard ReLU with a discontinuous step function at a learned threshold, optimizing an L_0 penalty to achieve the same decoupling. TopK SAEs (Gao et al., 2025) replace the sparsity penalty by explicitly retaining only the k highest pre-activations per token. BatchTopK SAEs (Bussmann et al.,

Table 2. Comparison of some sparse autoencoder (SAE) variants.

Type	Description
Standard ReLU (Bricken et al., 2023a)	Applies an L_1 penalty to hidden activations using a standard ReLU activation function to encourage sparsity.
Gated (Rajamanoharan et al., 2024a)	Decouples feature detection from magnitude estimation using a gated mechanism to prevent the shrinkage issues caused by standard L_1 penalties.
JumpReLU (Rajamanoharan et al., 2024b)	Uses a discontinuous step function at a learned per-feature threshold, approximating an L_0 penalty through straight-through estimation. Notably used in (Lieberum et al., 2024).
BatchTopK (Bussmann et al., 2024)	Retains top $n \times k$ activations over a batch of n inputs and sets the rest to zero, where k is a tuned hyperparameter. This forces an average of k latent activations per input, so the actual L_0 sparsity is approximately k. Notably used in (Brixi et al., 2026; Karvonen et al., 2025).
Matryoshka (Bussmann et al., 2025)	Trains nested sub-dictionaries of increasing size that share a single encoder/decoder, producing multiple resolutions of feature granularity in one run. Notably used in (McDougall et al., 2025).

2024) extend this by enforcing the TopK constraint across an entire batch, allowing natural variation in the number of active latents per token while maintaining the target sparsity in expectation. Matryoshka SAEs (Bussmann et al., 2025) train nested sub-dictionaries of increasing size under a shared encoder and decoder, producing representations at multiple levels of granularity from a single training run. We use Matryoshka, BatchTopK and JumpReLU variants in this paper.

SAE performance is typically evaluated as the pareto frontier of two metrics: L_0 norm of the SAE latent activations vector, which measures sparsity, and reconstruction quality (explained variance, cross-entropy increase when patching reconstructions into the model) (Karvonen et al., 2025; Ferrando et al., 2024; Bricken et al., 2023b).

SAEs have been successfully used on exploratory interpretability tasks like model auditing (Marks et al., 2025) and hypothesis generation (Movva et al., 2025), but it is unclear whether they can be used to recover the true features of models (Leask et al., 2025a; Chanin et al., 2026). For example, Leask et al. (2025a) show that SAEs can achieve high reconstruction quality while learning latents that do not correspond to any interpretable unit of computation. Therefore, reconstruction metrics are necessary but not sufficient evidence that an SAE has recovered the model's true representational structure.

Wattenberg & Viégas (2024) argue that matrix binding will result in echo features in SAEs, where the SAE learns latents corresponding to projections of the same feature into different binding subspaces. The effects of ID vectors and onion representations on SAEs have so far not been studied. SAE latents are generally treated as binary features in the literature (Paulo et al., 2025; Quirke et al., 2025), where they are 'on' if the feature is active and 'off' otherwise. In contrast, our results demonstrate graded latent activations in which relative activation magnitude encodes relational information, such as sequence order. If these results generalise to LLMs, then they could undermine this binary perspective. Additionally, graded latent activations may be ignored or misinterpreted by any downstream interpretability tool that assumes all features are independently interpretable and linear.

3. Methodology

We hypothesise that relational composition occurs at the `SEP` position's residual stream activation vector after the first layer of the two-layer transformer model, as implied by Farrell et al. (2025). We refer to this vector as $\boldsymbol{r}_s^{L_1}$. Therefore, we train SAEs on $\boldsymbol{r}_s^{L_1}$ to analyse how they respond to relational composition, as shown in Figure 1.

There are many SAE variants in the literature, but we consider three: BatchTopK (Bussmann et al., 2024), JumpReLU (Rajamanoharan et al., 2024b) and Matryoshka (Bussmann et al., 2025). A comparison is given in Table 2. These have reliable open-source implementations in the `dictionary_learning` library (Marks et al., 2024), and they represent the state of the art: Matryoshka and JumpReLU SAEs have been used by Google for their open suite of interpretability tools (Lieberum et al., 2024; McDougall et al., 2025), and all three are highly cited.

It is not feasible to analyse every SAE under every reason-

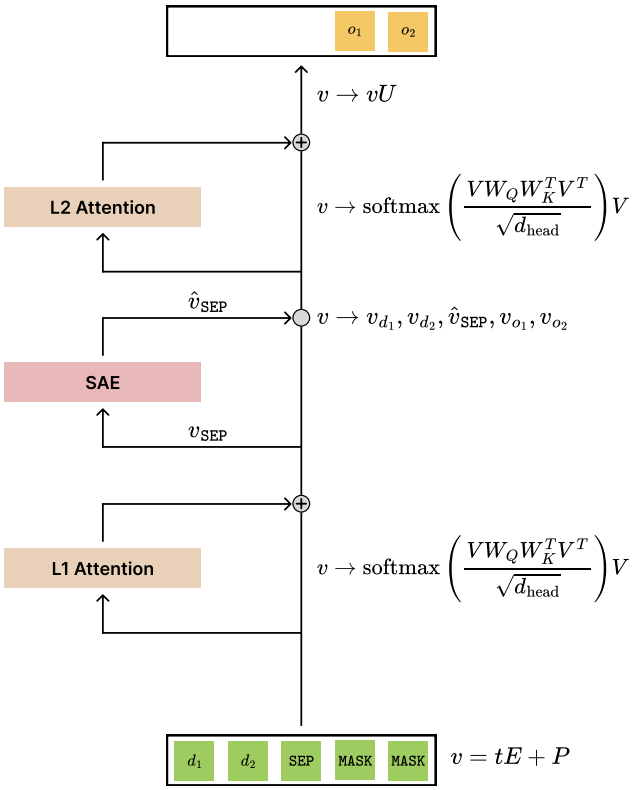

Figure 1. This diagram shows the structure of the two-layer attention-only transformer and how the SAE is used to extract and analyse the learned representations. v denotes the residual stream, which is read from and written to after each layer and has shape $(5, d_{\text{model}})$. t is the vector of input tokens $[d_1, d_2, \texttt{SEP}, \texttt{MASK}, \texttt{MASK}]$, E and P are the token and position embedding matrices respectively, U is the unembedding matrix, and each attention layer has key and query matrices, $\boldsymbol{K}^i$ and $\boldsymbol{Q}^i$ respectively, for $i \in [1, 2, 3]$. The value and output matrices for each layer are fixed to the identity, and $d_{\text{head}} = 64$. The SAE takes the residual vector at $\texttt{SEP}$, $v_{\texttt{SEP}}$, as input and patches reconstruction $\hat{v}_{\texttt{SEP}}$ back into the stream. Accuracy is calculated on the final two output logits only, represented by (o_1, o_2).

able hyperparameter configuration, so we select a single BatchTopK SAE for in-depth study. We validate key findings across 1,630 diverse SAE configurations and seeds (Table 3), as well as across different SAE architectures (BatchTopK, JumpReLU, Matryoshka), demonstrating that our results are not an artefact of this specific checkpoint. We select BatchTopK for its simplicity: it works by keeping the top $n \times k$ activations over a batch of n inputs and sets the rest to zero, where k is a tuned hyperparameter. This forces an average of k latent activations per input, so the actual L_0 sparsity is approximately k. In contrast, Matryoshka SAEs work similarly but force a hierarchical dictionary, and JumpReLU SAEs use the more complex JumpReLU activation function. However, by generating hypotheses based on the in-depth BatchTopK analysis and then testing them against the more complex variants, we support generalisation of our results.

Table 3. SAE hyperparameter sweep configurations. Braces denote swept values; fixed values are listed directly. Fixed parameters across all runs: 50,000 training steps, 1,000 warmup steps, 4,096 batch size. Total run counts are the product of all swept values and seeds. Note that $n_{\text{groups}} = 1$ in a Matryoshka SAE reduces it to a standard BatchTopK SAE, so this serves as an internal consistency check.

Shared Parameters	**Swept Values**
d_{SAE} (All)	{128, 192, 256, 320}
d_{SAE} (BatchTopK extra)	{64, 384}
k / Target L_0 (All)	{3, 4, 5}
k / Target L_0 (BatchTopK extra)	{1, 2}
k / Target L_0 (JumpReLU extra)	{2}
Learning rate (BatchTopK, Matryoshka)	3×10^{-4}
Architecture-Specific	**Values**
JumpReLU Sparsity penalty	{0.1, 0.5, 1.0, 5.0}
JumpReLU Learning rate	$\{7\times10^{-5}, 3\times10^{-4}\}$
Matryoshka n_{groups}	{1, 2, 4}
Experiment Scale	**Seeds / Total Runs**
BatchTopK	15 seeds / 450 runs
JumpReLU	5 seeds / 640 runs
Matryoshka	15 seeds / 540 runs

To select a specific BatchTopK SAE checkpoint to analyse, we run a grid-search over $k \in [1, 5]$ and SAE expansion factors $i \in [1, 6]$, such that $d_{\text{sae}} = i \times d_{\text{model}}$. Results are shown in Table 4. We compare SAEs based on:

1. **Explained variance (EV)**: variance of the input explained by the SAE's reconstruction is a popular and robust measure for the SAE's reconstruction quality (Bussmann et al., 2024; McDougall et al., 2025).
2. **Patched cross-entropy loss (PCE)**: cross-entropy loss when patching the SAE-reconstructed activations back into the model. This is an effective measure for how the SAE impacts downstream performance, because perfect SAE reconstructions should solve the task as effectively as the model. As this reflects reconstruction quality, it is a complementary metric to EV. The baseline model cross-entropy loss was 0.1115, so PCE $= 0.1115$ is optimal.
3. **Sparsity (L_0)**: sparser SAEs (smaller L_0 norm) have fewer latents activate for each input, which generally means the learned latents are more interpretable (Bricken et al., 2023b; Karvonen et al., 2025).
4. **Percentage of dead (inactive) latents**: SAEs often have some dead latents which do not activate for any input. A high percentage indicates that the SAE is not efficiently using its capacity. We expect to see this increase as d_{SAE} increases. While this is a useful metric for general SAE quality, dead latents are discarded

Table 4. SAE sweep results aggregated over seeds and learning rates (mean ± std). L_0: mean active features per token. d_{SAE}: dictionary size. **EV** (explained variance): fraction of activation variance recovered by the SAE reconstruction (↑ better). **PCE** (patched cross-entropy): language-model CE after substituting SAE reconstructions for true activations (↓ better; baseline = 0.1115). **Dead %**: percentage of dictionary features with zero activation across the evaluation set (↓ better). **Bold**: best within each L_0 block; **underlined**: best overall.

L_0	d_{SAE}	EV (↑)	PCE (↓)	Dead % (↓)
1	64	0.5946 ± 0.0035	7.5576 ± 0.2640	$\mathbf{0.1 \pm 0.4}$
1	128	0.6639 ± 0.0113	5.6684 ± 0.7471	11.7 ± 5.9
1	192	0.6668 ± 0.0089	5.4586 ± 0.5238	38.7 ± 2.8
1	256	0.6670 ± 0.0074	5.3517 ± 0.3238	49.2 ± 4.8
1	320	$\mathbf{0.6709 \pm 0.0065}$	5.4254 ± 0.3393	56.7 ± 5.8
1	384	0.6683 ± 0.0066	$\mathbf{5.1292 \pm 0.2389}$	57.8 ± 7.9
2	64	0.7396 ± 0.0062	2.6535 ± 0.1888	**0.0**
2	128	0.9351 ± 0.0049	1.9155 ± 0.2133	15.0 ± 3.2
2	192	0.9375 ± 0.0086	1.7859 ± 0.1338	37.3 ± 5.6
2	256	0.9347 ± 0.0063	1.7910 ± 0.2067	49.9 ± 8.7
2	320	$\mathbf{0.9377 \pm 0.0066}$	1.8507 ± 0.1693	55.2 ± 11.4
2	384	0.9318 ± 0.0089	$\mathbf{1.7846 \pm 0.1073}$	59.4 ± 12.8
3	64	0.7874 ± 0.0049	1.3783 ± 0.1077	**0.0**
3	128	0.9922 ± 0.0007	0.1853 ± 0.0118	10.9 ± 5.3
3	192	$\mathbf{0.9924 \pm 0.0006}$	$\mathbf{0.1765 \pm 0.0095}$	35.7 ± 7.0
3	256	0.9763 ± 0.0697	0.3261 ± 0.6377	40.3 ± 12.4
3	320	0.9915 ± 0.0017	0.1900 ± 0.0433	45.3 ± 14.3
3	384	0.9916 ± 0.0017	0.1914 ± 0.0173	52.7 ± 12.2
4	64	0.8135 ± 0.0077	0.9146 ± 0.1187	**0.0**
4	128	0.9921 ± 0.0004	$\mathbf{0.1799 \pm 0.0044}$	2.5 ± 3.5
4	192	$\mathbf{0.9921 \pm 0.0008}$	0.1826 ± 0.0056	14.3 ± 11.5
4	256	0.9920 ± 0.0013	0.1871 ± 0.0058	19.5 ± 11.9
4	320	0.9918 ± 0.0016	0.1983 ± 0.0263	27.2 ± 16.2
4	384	0.9913 ± 0.0021	0.2023 ± 0.0191	35.2 ± 14.1
5	64	0.8327 ± 0.0064	0.7296 ± 0.0937	**0.0**
5	128	0.9921 ± 0.0002	$\mathbf{0.1750 \pm 0.0023}$	1.6 ± 1.8
5	192	$\mathbf{0.9924 \pm 0.0007}$	0.1826 ± 0.0054	10.2 ± 8.9
5	256	0.9908 ± 0.0012	0.1984 ± 0.0117	9.7 ± 9.1
5	320	0.9916 ± 0.0014	0.2124 ± 0.0203	16.4 ± 17.3
5	384	0.9894 ± 0.0015	0.2275 ± 0.0187	15.9 ± 13.5

during analysis so this metric is not as critical as the others.

We compute the metrics over the combined training and test dataset consisting of all 10,000 possible input bigrams. This provides more data than just the 2000 test examples which improves the generalisability of our results. Since the SAE is trained on model activations rather than inputs, there is no need for test-train splits. The ideal outcome is the SAE overfits to the target model latent's behaviour, so we can study the SAE and more robustly generalise to the model latent (although we cannot generalise with full confidence as discussed in Section 2 (Chanin et al., 2026; Leask et al., 2025a)).

Our grid-search shows that BatchTopK SAEs with $k = 1$ and $k = 2$ perform much worse than other SAEs for both PCE and EV. BatchTopK SAEs with $k \in [3, 5]$ are joint-best for EV up to 3 s.f. and PCE up to 2 s.f. We take $k = 3$ because performance difference is negligible and lower sparsity typically improves interpretability and analysis: with at most 3 latents active per token, we can more easily exhaustively analyse each latent's role in any given computation.

SAEs with $d_{\text{sae}} \in [128, 320]$ perform best for EV and PCE. $d_{\text{sae}} = 64$ performs poorly because the transformer has 64 dimensions which are likely polysemantic, making it difficult to expand each model dimension into monosemantic SAE latents. We show this result to demonstrate the importance of SAE dimension choice. Conversely, when d_{sae} is too large, dead latents increase sharply (Table 4), indicating that the dictionary size exceeds what the model's representations actually need to be represented monosemantically.

Table 5. Selected SAE configuration. The learning rate, optimiser, betas, schedule and auxiliary loss are defaults from (Bussmann et al., 2024; Marks et al., 2024) and were constant for all BatchTopK runs.

Parameter	**Value**
SAE type	BatchTopK (Bussmann et al., 2024)
Input dimension (d_{model})	64
Dictionary size (d_{SAE})	128
Expansion factor	$128/64 = 2\times$
k	3
Actual sparsity (L_0)	3.00
Learning rate	3×10^{-4}
Optimiser	`Adam`
Betas	$(0.9,\ 0.999)$
Batch size	4096
Training steps	50,000
LR schedule	Linear warmup (1,000 steps), then constant
Auxiliary loss (α)	$1/32$
Seed	1
Hardware	NVIDIA GeForce RTX 2080 Ti

We therefore study a BatchTopK SAE with $k = 3$ and $d_{\text{SAE}} = 128$, as shown in Table 5. Among the trained checkpoints with this configuration, we select the one with the fewest dead latents (3.1%), which is far below the $k = 3, d_{\text{SAE}} = 128$ group average of $10.9 \pm 5.3\%$, and achieves 0.9919 EV and 0.1796 PCE. This checkpoint selection introduces selection bias toward high-performing models. We address this by confirming across 1,630 diverse SAEs that special latents (identifiable by $\Delta\alpha$ correlation) emerge reliably regardless of configuration, establishing that our findings are not specific to this checkpoint.

3.0.1. Analysis

We can sometimes learn what feature an SAE latent has learned to represent by looking at inputs that cause it to activate (Bills et al., 2023), so we begin with this approach. This allows us to observe, for each latent, the sort of inputs that it responds to and we can generate hypotheses based

on correlations between latent activation and various input properties. We test these hypotheses using latent steering (Ferrando et al., 2024; Arditi et al., 2024) in which we linearly scale the SAE latent's activation for a given input, patch the SAE reconstruction back into the model downstream and compare the new output to the original output. If the output changes predictably according to the hypothesis for what that latent represents, then that supports the hypothesis (Section 2). Since the entire input space is small (10,000 possible input bigrams), we can feasibly record SAE activations for every single input, which is more robust than sampling activations and inferring to other inputs.

To test whether our results on the selected SAE checkpoint generalise to others, we check for similar results on 1630 different SAE configurations and seeds (Table 3) where feasible. When this is computationally infeasible, we use a targeted subset of SAE checkpoints as shown in Table 6, which were selected for strong performance and varying d_{SAE} and L_0 to provide a diverse test sample, although we note that this small set is not sufficient for statistical significance.

4. Results

4.1. Analysing Features

For the selected BatchTopK SAE checkpoint (Table 5) trained on the residual at `SEP`, $\boldsymbol{r}_s^{L_1}$, we inspect the learned latents and their activations for different input bigrams. We plot a heatmap for each latent to provide a visual overview of the bigrams that activate it, as shown in Figure 2.

In this way, we identify two distinct types of latents that this SAE has learned (excluding 8% of latents, which activate on fewer than 0.1% of inputs): k-symbol detectors and special latents.

k-symbol detectors (91% of latents): 88% of latents are 1-symbol detectors: they activate only if one specific symbol is present in the input at either d_1 or d_2, shown by the 'plus-sign' pattern in Figure 2. They activate most strongly where $d_1 = d_2$ (the intersection), and otherwise their activation magnitude varies only with symbol presence, not position. This aligns with the binary view of SAE latents: they represent a single feature (symbol presence) and activate if that feature is present. 3% of latents detect multiple symbols ($k > 1$), such as latent 5 in Figure 2. Moreover, these sometimes activate with different magnitudes for different symbols (e.g., latent 5 activates stronger for symbol 7 than symbol 58), suggesting magnitude may carry information.

Special latents (2 latents, 9% of activations): Only two latents (11 and 56) do not follow the above pattern. Latent 11 activates on 64% of inputs and latent 56 on 9.9%, compared to ¡4.5% for all other latents (Appendix A). Because they activate across most inputs, their activation cannot indicate whether a specific symbol is present.

Moreover, our earlier circuits analysis shows that both representations for input bigrams (a, b) and (b, a), $a, b \in [0, 99]$, at $\boldsymbol{r}_s^{L_1}$ are linear combinations of the input token and positional embeddings with coefficients defined by the first layer's attention pattern. Therefore, we investigate whether latents' activation magnitudes correlate with the relative magnitude of the `SEP` position's attention to d_1 and d_2, $\Delta\alpha = \alpha_{s\to d_1} - \alpha_{s\to d_2}$, as shown in Figure 3. Latent 11's activation magnitude strongly positively correlates with $\Delta\alpha$ (Pearson correlation coefficient $r = 0.807$), and latent 56 weakly negatively correlates with $\Delta\alpha$ ($r = -0.3213$). We compute correlations over the whole dataset, and all latents have correlation $|r| < 0.004$ with $\Delta\alpha$, suggesting that these two features are indeed special relative to the others. Since latent 11 strongly positively correlates with $\Delta\alpha$ we refer to it as strongly d_1-favouring, and we similarly refer to latent 56 as weakly d_2-favouring. Latent 11 has a much stronger correlation than 56, so we refer to it as the primary special feature.

4.2. Steering Experiments

We hypothesise that special latents could have graded activations, which would mean they exhibit different behavior at different activation magnitudes, rather than simply being on/off. Specifically, we hypothesise that these special latents have meaningful activation ranges corresponding to different bigram orderings. To test this hypothesis, we use steering vectors (Ferrando et al., 2024; Arditi et al., 2024) to perform a causal linear intervention: we scale special latent activations and observe whether the model's outputs change predictably. More specifically, we:

1. Input bigram (d_1, d_2) into the transformer model (Figure 1) and record the special latent's activation magnitude m for that input, using the SAE trained on $\boldsymbol{r}_s^{L_1}$.
2. Scale m by some factor p and reconstruct $\boldsymbol{r}_s^{L_1}$ as $\hat{\boldsymbol{r}}_s^{L_1}$ using the modified SAE activations.
3. Replace $\boldsymbol{r}_s^{L_1}$ with $\hat{\boldsymbol{r}}_s^{L_1}$ in the model, and compute the logits at positions (o_1, o_2) as normal.
4. Repeat for all factors $p \in [0, 10]$ with step size 0.05 across all input bigrams. Predictable and consistent shifts in the output logits under this intervention provide evidence that the special latent is graded.

The binary view (Quirke et al., 2025) expects SAE latents to activate when a feature is present and remain inactive otherwise, with magnitude being irrelevant to the feature represented. If this were true for special latents, scaling their activation (step 2 above) would either keep outputs

Table 6. Comparison of cherry-picked SAE architecture types across configuration and performance metrics. These SAEs are used to support generalisation claims when a large testing sample is not computationally feasible.Dashes indicate parameters not applicable to a given architecture. L_0 (actual) is the mean number of active latents measured at eval time.

SAE Type	Configuration						Performance			
	d_{SAE}	LR	k	Target L_0	Sp. Pen.	n_{groups}	CE Increase ↓	Expl. Var. ↑	Dead (%) ↓	L_0 (actual)
BatchTopK	192	3×10^{-4}	3	—	—	—	0.0257	0.9929	26.04	3.36
	128	3×10^{-4}	5	—	—	—	0.0292	0.9939	0.78	4.90
JumpReLU	256	7×10^{-5}	—	5	5	—	0.0141	0.9965	50.78	4.03
	128	7×10^{-5}	—	5	5	—	0.0176	0.9963	17.97	3.24
Matryoshka	256	3×10^{-4}	3	—	—	2	0.0262	0.9928	39.06	3.32
	192	3×10^{-4}	4	—	—	4	0.0325	0.9927	12.50	3.98

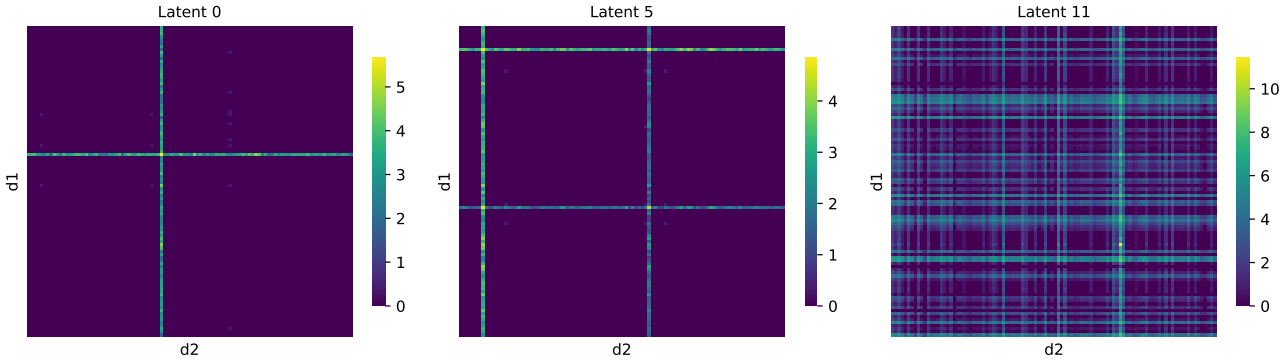

Figure 2. These 100×100 heatmaps show how specific SAE latents activate for different input bigrams (d_1, d_2), of which there are 100×100 combinations. The top-left corner cell is input $(0, 0)$ and the bottom-right corner cell is $(99, 99)$. Each heatmap shows: **Left** (1-symbol detector): latent 0 activates only if d_1 or $d_2 = 41$, and activates most strongly when $d_1 = d_2 = 41$. This results in a 'plus-sign' pattern. **Center** (k-symbol detector, $k > 1$): latent 5 activates only if d_1 or d_2 is either 7 or 58, resulting in two 'plus-sign' patterns. **Right** (special latent): latent 11 activates for most inputs and activates much more strongly at peak than all other latents.

unchanged (same latents remain active) or produce nonsense. However, based on the circuits analysis in Farrell et al. (2025) showing that output order depends on $\Delta\alpha$, and the observation that special latents correlate with $\Delta\alpha$ (Section 4.1), we predict that scaling these latents will cause systematic output swaps: for input (a, b) where the model correctly outputs (a, b), scaling the special latent may produce output (b, a) at appropriate scaling factors.

Figure 4 shows a specific example input for which the predicted behaviour occurs using latent 11. When no latent scaling is applied ($p = 1$) we see that the argmax at o_1 matches the input token at d_1 as expected, and similarly o_2 matches d_2. However, for $p \in [0.16, 2.20]$ we have the opposite: o_1 has argmax d_2 and o_2 has argmax d_1. In this way, latent 11 behaves as a graded latent for this input. We define a successful output swap as follows: the symbol from d_1 is predicted at position o_2 and the symbol from d_2 is predicted at position o_1. The 'swap range' is the set of scaling factors p for which this condition holds.

We test all 10,000 input bigrams and find that steering latent 11 produces output swaps for 5,002 out of 10,000 input bigrams (50.0%). For 3,600 inputs (36%), latent 11 does not activate, so steering has no effect ($m = 0$). For the remaining 1,424 inputs (14.2%), latent 11 activates but no scaling produces a swap. This occurs for several reasons (visual examples in appendices in the code submission):

- **No positive overlapping scale ranges**: for 9.2% of inputs, latent 11 must be negatively scaled to swap the output at one of the output positions. BatchTopK SAE latent activations are strictly non-negative, where zero indicates feature absence and positive values indicate feature presence. Scaling a latent negatively does not represent absence, but rather introduces an out-of-distribution 'anti-feature' by subtracting its vector from the residual stream. We reject these cases because they force the SAE to behave in a way that is mathematically impossible during training, meaning they cannot be used as evidence for a naturally learned graded latent.

- **Positive but mutually exclusive scale ranges**: for 2.3% of inputs, it is possible to scale the latent to swap the symbol at each output position individually but not

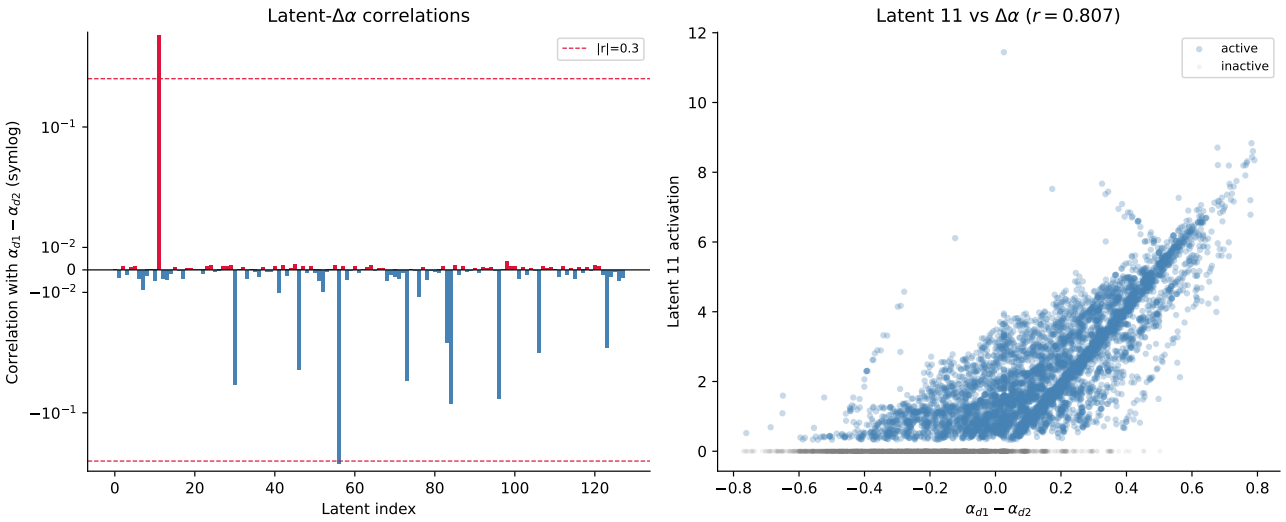

Figure 3. **Left**: This plot shows the (Pearson) correlation of each latent with $\Delta\alpha = \alpha_{s\to d_1} - \alpha_{s\to d_2}$, which is intuitively the emphasis given to d_1 over d_2 by SEP in the transformer model's first layer's attention pattern. The y-axis uses a symmetric logarithmic scale with a linear scale between $[-0.05, 0.05]$. d_1-favouring latents are highlighted red and d_2-favouring latents are highlighted blue. The top three strongest correlations are: latent 11 with $r = 0.807$, latent 56 with $r = -0.3213$, and latent 96 with $r = 0.0036$. **Right**: This plot shows the correlation of latent 11 with $\Delta\alpha$ in more detail, as this latent has the strongest correlation. All 10,000 possible input bigrams are plotted, and highlighted blue if they activate latent 11 and grey otherwise. We see that the latent activates with greater magnitude for larger $\Delta\alpha$.

simultaneously.

- **No swap occurs in observed ranges**: for 1.6% of inputs, the logit for the symbol at d_2 is always dominant at o_2 in the observed scale range, so there is no scale where the the symbol at d_1 is predicted instead. For 0.2% of inputs a similar situation occurs at o_1.
- **Matching bigram elements**: 0.8% of the inputs both activate the latent and have identical input symbols, so swapping the outputs is degenerate. The remaining 0.2% of same-symbol inputs do not activate the latent.

In limited experiments with inputs that fail for the first three causes, it is sometimes possible to scale a co-activating (non-special) latent to successfully swap outputs. For example, input $(41, 49)$ activates latents 11 (magnitude 3.1), 0 (magnitude 3.1) and 55 (magnitude 2.5). The swap fails with latent 11 because there are no positive overlapping scale ranges; however, scaling latent 55 by $p \in [1.1, 1.15]$ successfully produces a swap. This does not necessarily indicate that latent 55 is graded; rather, scaling it alters the logit distribution, which opens a valid swap zone for latent 11. Similarly, running the steering pipeline on latent 55 alone swaps only 4 inputs, all of which co-activate latent 11, suggesting a similar mechanism. We note this preliminary experiment is insufficient for statistical significance, but it suggests that scaling multiple latents together may enable more output swaps than steering special latents alone. Detailed visualisations are provided in the appendices.

Table 7. We perform steering experiments on several chosen and randomly sampled SAE latents to demonstrate how steering with special latents (**bold**) compares to non-special latents. We record how many outputs are successfully swapped by steering only that latent, how many do not activate the latent, and how many activate the latent but cannot be swapped.

Latent	Swapped	No Activation	Active & No Swap
0	6 (0.1%)	9786 (97.9%)	208 (2.1%)
5	23 (0.2%)	9594 (95.9%)	383 (3.8%)
11	**4976 (49.8%)**	**3600 (36.0%)**	**1424 (14.2%)**
55	4 (0.0%)	9787 (97.9%)	209 (2.1%)
56	**196 (2.0%)**	**9009 (90.1%)**	**795 (7.9%)**
63	4 (0.0%)	9801 (98.0%)	195 (2.0%)
82	4 (0.0%)	9787 (97.9%)	209 (2.1%)
117	37 (0.4%)	9782 (97.8%)	181 (1.8%)

We additionally steer latent 56 and successfully swap 2.0% of outputs, which is part of the 36% of inputs that do not activate latent 11 since the two special latents never coactivate.

We do not attempt steering on all latents due to computational constraints, but we attempt it on five randomly selected latents (0, 55, 63, 82, 117) and latent 96 (non-special latent with highest $\Delta\alpha$ correlation, $r = 0.0036$). Results are shown in Table 7. We find that every single successful swap using a non-special latent has a co-activating special latent, which suggests that these non-special latents are un-

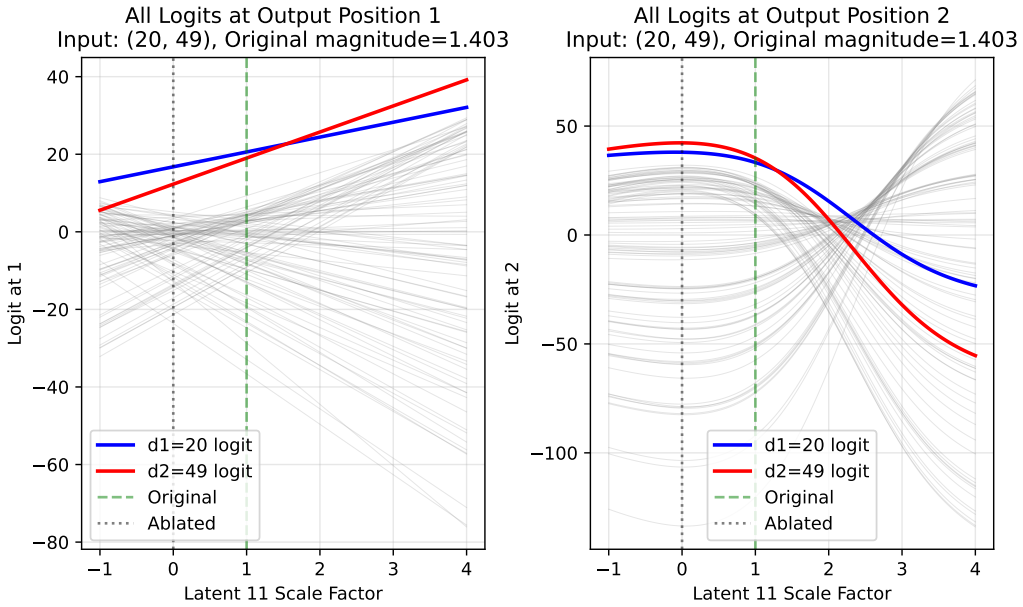

Figure 4. This graph shows for input $(20, 49)$ how the logits at the two output positions change as latent 11's activation magnitude is scaled. Each line shows the logit for some $i \in [0, 99]$ (100 lines total), such that if the top line at some latent scale represents symbol i, then i is predicted at that output position when the latent's activation magnitude is scaled by that factor. The line corresponding to the symbol in position d_1 is blue, and the symbol in position d_2 is red. For the outputs to swap from $(20, 49)$ (correct) to $(49, 20)$ by scaling the latent's activation magnitude, there must be a range of scales where the red line is on top on the left plot and the blue line is on top on the right plot. For this example, this range is $[0.16, 2.20]$.

likely to be graded, otherwise they could singlehandedly swap more outputs, and instead alter the logit distribution such that the graded special latents can successfully swap outputs.

4.3. Generalisation to other SAEs

To confirm these findings are not a quirk of this specific SAE, we observe whether they occur across other SAEs.

To achieve this at scale, we automatically identify special latents by checking each latent's correlation with $\Delta\alpha$ is over threshold $|r| > 0.3$ (Figure 3). Figure 5 shows that SAEs with strong performance reliably learn special latents. This strongly suggests that the behaviour described so far in this section generalises to various SAE configurations. However, this generalisation is limited because we only tested three SAE types (BatchTopK, Matryoshka, JumpReLU) over 15 seeds. Future work should check for generalisation to more SAE variants such as those in Table 2, and over more training seeds for more robust statistical significance testing.

To test whether special latents can be steered to swap outputs in other SAEs, we provide the following pipeline that works given any SAE, which is available in our code repo:

1. Store the SAE activations for all input bigrams by running a single forward pass as in Figure 1 over the full dataset and recording each SAE latent's activation magnitude.
2. Identify special latents using $\Delta\alpha$ correlation $|r| > 0.3$ as a heuristic, as described previously.
3. For each identified special latent, run a batched steering grid search over all input bigrams. For each bigram and each scale factor $p \in [0, 10]$ with step size 0.05 (201 values), record the logits at output positions o_1 and o_2 under the steered reconstruction $\hat{\boldsymbol{r}}_s^{L_1}$. Bigrams for which the latent does not activate ($m = 0$) are skipped immediately since scaling has no effect.
4. Find crossover scales for each bigram:
   - For o_1: since o_1 logits are empirically linear in p, fit lines to $\ell_{d_1}(p)$ and $\ell_{d_2}(p)$ and compute the analytical intersection. Inputs for which either logit series fails an $R^2 \geq 0.999$ linearity check, or for which the intersection falls on a negative scale, are flagged and excluded.
   - For o_2: detect sign changes in $\ell_{d_1}(p) - \ell_{d_2}(p)$ on the coarse $[0, 10]$ grid, then refine each crossing to three decimal places using bisection search. All bisection tasks across the batch are executed in parallel to avoid sequential per-input forward passes.
5. Determine a valid swap zone $[p_{\text{lo}}, p_{\text{hi}}]$ per bigram by intersecting the o_1 and o_2 crossover constraints with

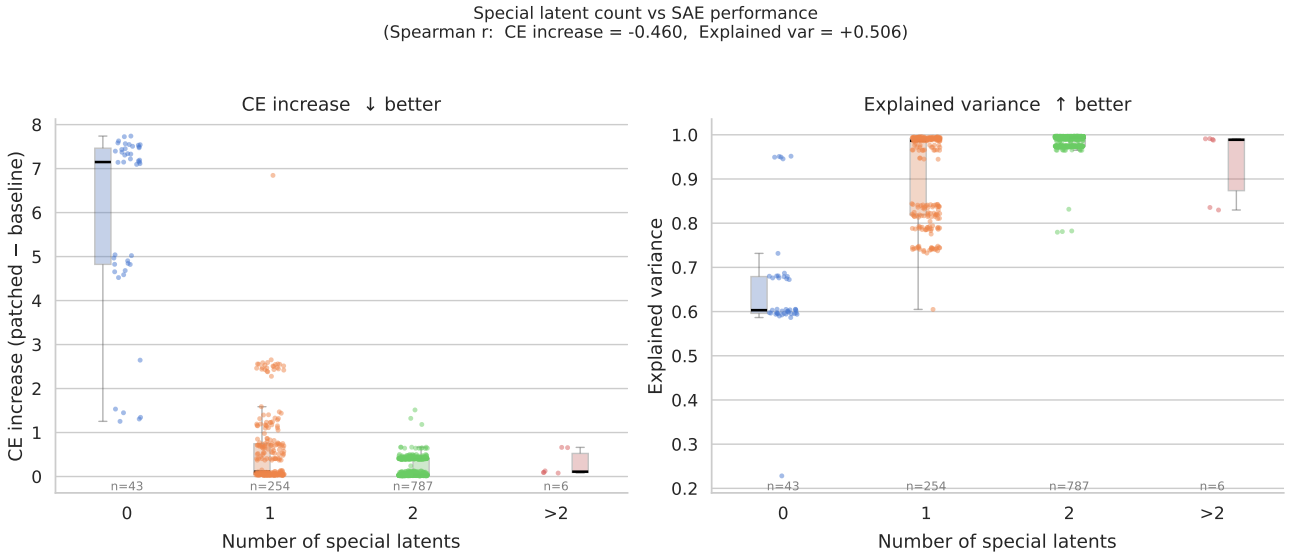

Figure 5. These strip plots show how the number of special latents (identified by the latent's activation's correlation with $\Delta\alpha$ using threshold $|r| > 0.3$, as in Section 4.1) differs across SAEs with different performance. It shows that SAEs with strong performance, measured by explained variance and patched cross-entropy loss, tend to have 2 special latents (spearman $r = -0.460$ for patched cross-entropy loss and $r = 0.506$ for explained variance).

the scale ranges over which $\text{argmax}(o_1) = d_2$ and $\text{argmax}(o_2) = d_1$ hold simultaneously. If no contiguous window satisfying both conditions exists, the bigram is recorded as a failure with a specific reason (see Section 4.2).

6. Verify each swap zone by running the model at the midpoint $p^* = (p_{\text{lo}} + p_{\text{hi}})/2$ and confirming that the model output is (d_2, d_1).

7. Report the fraction of bigrams for which a valid swap zone was found and the swap verified, along with a breakdown of failure reasons for the remainder.

Steps 3-5 of the pipeline are very computationally expensive, so we test generalisation using a varied sample of six high performing SAEs (Table 6) rather than running the pipeline on all trained SAEs (as in Figure 5). The results are shown in Table 8. We find that all six sampled SAEs learned at least one special latent ($|r| > 0.3$), and five learned a pair of opposing special latents (one d_1-favouring and one d_2-favouring). In every tested SAE, steering at least one of these special latents successfully swapped the predicted outputs for a substantial portion of the input space, ranging from 26.6% to 72.5% of all inputs. When the output swap failed, the inputs typically either did not activate the latent at all or encountered constraints similar to those detailed in Section 4.2, such as requiring negative activation scaling. This suggests that the relative magnitude-based mechanism, and the emergence of steerable, graded latent activations, generalizes consistently across varying SAE architectures and hyperparameter configurations. In future work, this study should be extended to further configurations for robust statistical significance.

5. Discussion

Farrell et al. (2025) introduce a toy transformer that solves ordered list copying by writing both inputs into the `SEP` position and then decoding order by the relative magnitudes of the contributions (the token and positional embeddings of the inputs), rather than projecting information into role-specific directions as is the case in additive matrix binding (Wattenberg & Viégas, 2024).

Consider training an L_1-regularised SAE on the residual at `SEP` after layer 1,

$$\boldsymbol{r}_s^{L_1} = \alpha_{s\to d_1}(\boldsymbol{E}_a + \boldsymbol{P}_{d_1}) + \alpha_{s\to d_2}(\boldsymbol{E}_b + \boldsymbol{P}_{d_2}) + \boldsymbol{E}_s + \boldsymbol{P}_s$$

This is the transformer model's internal single fixed-length vector representation of input bigram (a, b). Here, we have a subspace spanned by $\boldsymbol{D}_a = \boldsymbol{E}_a + \boldsymbol{P}_{d_1}$ and $\boldsymbol{D}_b = \boldsymbol{E}_b + \boldsymbol{P}_{d_2}$ which is specific to two input bigrams, (a, b) and (b, a), as $\boldsymbol{E}_s + \boldsymbol{P}_s$ is constant for all inputs.

In theory, the SAE could learn a latent for each bigram, which activates when that bigram appears in the model's input. Alternatively, the SAE could learn latents corresponding to $\boldsymbol{D}_a$ and $\boldsymbol{D}_b$ with activations equal to the attention probabilities α. Since the latter latents can be reused for other bigrams containing a or b, the latter solution always

Table 8. Generalisation of special latent steering to other SAE configurations from Table 6. For each identified special latent ($|r| > 0.3$), we report its bias, Pearson correlation r with $\Delta\alpha$, and the percentage of inputs where the latent was successfully steered to swap outputs (Success), did not activate (No Act.), or activated but could not be swapped (Active & No Swap).

SAE Type	d_{SAE}	L_0	Latent	Type	r	Steering Outcomes (%)		
						Success	No Act.	Active & No Swap
BatchTopK (Studied)	128	3.00	11	d_1-favouring	$+0.807$	49.8%	36.0%	14.2%
			56	d_2-favouring	-0.321	2.0%	90.1%	7.9%
BatchTopK	192	3.36	47	d_2-favouring	-0.836	52.9%	34.4%	12.7%
			116	d_1-favouring	$+0.365$	0.5%	91.7%	7.8%
	128	4.90	68	d_1-favouring	$+0.845$	32.9%	44.7%	22.4%
			64	d_2-favouring	-0.730	6.0%	62.0%	32.0%
JumpReLU	256	4.03	195	d_1-favouring	$+0.849$	38.8%	39.8%	21.4%
			179	d_2-favouring	-0.721	5.8%	60.2%	34.0%
	128	3.24	105	d_2-favouring	-0.816	28.6%	37.8%	33.6%
			4	d_1-favouring	$+0.756$	6.9%	62.3%	30.8%
Matryoshka	256	3.32	48	d_2-favouring	-0.796	26.6%	50.6%	22.8%
			62	d_1-favouring	$+0.591$	6.0%	71.2%	22.8%
	192	3.98	30	d_1-favouring	$+0.866$	72.5%	16.5%	11.0%

results in a lower average L_1 (sparsity) penalty than the former. In fact, this is the ideal solution since each input bigram can activate two latents corresponding to the symbols, with activation magnitudes corresponding the the bigram order. A sparser solution would require maximum one latent on average to activate for each input bigram, which necessarily results in a noisier reconstruction because that single latent would activate for all bigrams containing either a or b, hence it is not ideal.

This results in graded latent activations, as shown in Figure **??**, because the same two latents represent two opposite bigrams.

We support this prediction with a case study of a BatchTopK SAE in Section 4. We identify k-symbol detectors, which activate if one of k specific symbols appears anywhere in the input bigram. These are related to, but distinct from, the hypothesised $\boldsymbol{D}_a$ and $\boldsymbol{D}_b$ latents. The predicted $\boldsymbol{D}_a = \boldsymbol{E}_a + \boldsymbol{P}_{d_1}$ latent would be position-specific: it encodes symbol a at input position d_1, and would activate differently depending on whether a appears as d_1 or d_2. By contrast, k-symbol detectors activate whenever symbol a appears at either position and do not necessarily recover positional information. They therefore align with the binary view (Quirke et al., 2025; Paulo et al., 2025), activating if a symbol is present and not otherwise, and do not fully correspond to the theoretically predicted latents. We instead find that the SAEs learn latents corresponding to the difference in attention weights, $\Delta\alpha = \alpha_{s\to d_1} - \alpha_{s\to d_2}$, which exhibit graded activations for a significant number of inputs.

As such, the binary perspective of SAE latents (Quirke et al., 2025; Paulo et al., 2025), which claims that each latent activates if a feature is present and does not otherwise, is insufficient to explain the computation of this model, and the activation values of the SAE latents must be incorporated into our understanding. This creates separate problems to the echo-features caused by additive matrix binding (Wattenberg & Viégas, 2024), which are different directions in the activation space to the base features; and the ID features in ID binding (Feng & Steinhardt, 2024) which are also different directions but have an abstract interpretation. A feature that identifies the order of the bigram is necessarily not linear, due to the relative interactions between the input features. These contradict the linear representation hypothesis, which states that networks can be described in terms of independent and linear features (Elhage et al., 2022). Similarly to the results of Csordás et al. (2024) on RNNs, we provide evidence that transformer interpretability should not be confined by the linear representation hypothesis.

Recent work (Leask et al., 2025b; Chanin et al., 2026) has highlighted the practical challenges with training SAEs that are useful for mechanistic interpretability. Our results in this paper potentially cut across this debate: even if SAEs can recover the correct dictionary, without tools to understand the effect of the relative magnitudes of latent activations, our interpretation of the model will be incomplete.

6. Conclusion and Future Work

In this paper we identified a previously undescribed mechanism for relational composition in transformers, which we call relative magnitude-based composition: the model encodes bigram order through the relative sizes of scalar attention weights rather than through distinct directions in the activation space. We characterised this mechanism mathematically via circuit analysis, showed that SAEs trained on this model reliably learn graded special latents whose activation magnitudes correlate with $\Delta\alpha$, and demonstrated that steering these latents reverses the model's predicted output for over 50% of inputs. These results indicate that the activation values of SAE latents, and not only their active/inactive status, carry information that is necessary for understanding model behaviour in this setting, which is a different perspective on transformer features than is currently prevalent in the mechanistic interpretability literature. We release our trained SAEs and wandb training runs to support future work and reproducibility of our results.

Limitations

Our mechanism analysis is conducted on a single minimal toy model with a narrow, controlled task. The architecture omits components standard in pretrained LLMs (MLPs, LayerNorm, multi-head attention, value and output projections) and uses a vocabulary of only 100 symbols. However, our SAE experiments demonstrate robust findings across diverse configurations: special latents (characterized by $\Delta\alpha$ correlation) emerge across three distinct SAE architectures (BatchTopK, JumpReLU, Matryoshka) with varying sparsity and dictionary size settings 3. This suggests the mechanism may be fundamental to how SAEs represent the relational composition in this model, though whether relative magnitude-based composition appears in less constrained models remains an open question.

A further limitation is that our circuit analysis characterises the mechanism on inputs the model solves correctly. The roughly 9% of inputs on which the model errs seem to reflect cases where $\Delta\alpha \approx 0$ (as shown in Figure **??**), where the magnitude-based mechanism cannot resolve the two orderings. A complete account of the model's behaviour would need to explain these failure cases as well.

Additionally, the steering pipeline currently handles one special latent at a time; simultaneous multi-latent steering could cause even more outputs to be swapped, as suggested by the co-activation experiments in Section 4.2. Finally, our SAE checkpoint was selected on the basis of having few dead latents, which introduces selection bias toward high-quality SAEs. While generalisation experiments (Section 4.3) show key findings hold broadly, a fully representative study would require analysis across random samples of all trained SAEs.

Future Work

The most important next step is determining whether pretrained LLMs implement similar mechanisms. The GemmaScope suites of SAEs (Lieberum et al., 2024; McDougall et al., 2025) provide a natural test environment, as it includes JumpReLU and Matryoshka SAEs trained on Gemma 2 and Gemma 3 at multiple widths - the same SAE variants we study here. The specific target would be identifying SAE latents in pretrained models where activation magnitude, rather than feature presence, is causally relevant for downstream behaviour.

There are also several remaining extensions within the toy model setting. Scaling to larger N-grams would test how the minimum number of layers required for high accuracy scales with N, and whether the magnitude-based mechanism persists or is replaced by a direction-based alternative as the task becomes harder. Moreover, the role of positional encodings in supporting this mechanism is not fully understood, and it is unclear whether relative magnitude-based composition is possible under alternative positional encoding schemes such as absolute encodings (Radford et al., 2019) or RoPE (Su et al., 2024).

Furthermore, it is an open question as to whether the pathfinding models trained by Brinkmann et al. (2024), which were the inspiration for this paper, learn a similar relation composition mechanism.

On the SAE side, the steering pipeline could be extended in two directions: improving computational efficiency to allow exhaustive testing across all trained SAE configurations rather than a cherry-picked sample, and testing whether scaling multiple latents simultaneously can recover the remaining inputs for which single-latent steering fails. Preliminary evidence in Section 4.2 suggests this may be possible. Finally, the failure modes of the base transformer model (roughly 9% of input) have not been fully characterised, and it is worth investigating whether these relate systematically to the cases where latent steering also fails.

In summary, the findings in this paper are intended as a foundation for further research on magnitude-based relational composition mechanisms in LLMs.

Impact Statement

Authors are **required** to include a statement of the potential broader impact of their work, including its ethical aspects and future societal consequences. This statement should be in an unnumbered section at the end of the paper (co-located with Acknowledgements – the two may appear in either order, but both must be before References), and does not count toward the paper page limit. In many cases, where the ethical impacts and expected societal implications are

those that are well established when advancing the field of Machine Learning, substantial discussion is not required, and a simple statement such as the following will suffice:

"This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here."

The above statement can be used verbatim in such cases, but we encourage authors to think about whether there is content which does warrant further discussion, as this statement will be apparent if the paper is later flagged for ethics review.

References

Arditi, A., Obeso, O., Syed, A., Paleka, D., Panickssery, N., Gurnee, W., and Nanda, N. Refusal in language models is mediated by a single direction. *Advances in Neural Information Processing Systems*, 37:136037–136083, 2024.

Baeumel, T., Gurgurov, D., Al Ghussin, Y., Genabith, J. V., and Ostermann, S. Modular arithmetic: Language models solve math digit by digit. In Inui, K., Sakti, S., Wang, H., Wong, D. F., Bhattacharyya, P., Banerjee, B., Ekbal, A., Chakraborty, T., and Singh, D. P. (eds.), *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pp. 1380–1409, Mumbai, India, December 2025. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics. ISBN 979-8-89176-303-6. doi: 10.18653/v1/2025.findings-ijcnlp.86. URL https://aclanthology.org/2025.findings-ijcnlp.86/.

Bills, S., Cammarata, N., Mossing, D., Tillman, H., Gao, L., Goh, G., Sutskever, I., Leike, J., Wu, J., and Saunders, W. Language models can explain neurons in language models. https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html, 2023.

Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Askell, A., Lasenby, R., Wu, Y., Kravec, S., Schiefer, N., Maxwell, T., Joseph, N., Hatfield-Dodds, Z., Tamkin, A., Nguyen, K., McLean, B., Burke, J. E., Hume, T., Carter, S., Henighan, T., and Olah, C. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023a. https://transformer-circuits.pub/2023/monosemantic-features/index.html.

Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Askell, A., et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2, 2023b.

Brinkmann, J., Sheshadri, A., Levoso, V., Swoboda, P., and Bartelt, C. A mechanistic analysis of a transformer trained on a symbolic multi-step reasoning task. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 4082–4102, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.242. URL https://aclanthology.org/2024.findings-acl.242/.

Brixi, G., Durrant, M. G., Ku, J., Naghipourfar, M., Poli, M., Sun, G., Brockman, G., Chang, D., Fanton, A., Gonzalez, G. A., King, S. H., Li, D. B., Merchant, A. T., Nguyen, E., Ricci-Tam, C., Romero, D. W., Schmok, J. C., Taghibakhshi, A., Vorontsov, A., Yang, B., Deng, M., Gorton, L., Nguyen, N., Wang, N. K., Pearce, M. T., Simon, E., Adams, E., Amador, Z. J., Ashley, E. A., Baccus, S. A., Dai, H., Dillmann, S., Ermon, S., Guo, D., Herschl, M. H., Ilango, R., Janik, K., Lu, A. X., Mehta, R., Mofrad, M. R. K., Ng, M. Y., Pannu, J., Ré, C., St. John, J., Sullivan, J., Tey, J., Viggiano, B., Zhu, K., Zynda, G., Balsam, D., Collison, P., Costa, A. B., Hernandez-Boussard, T., Ho, E., Liu, M.-Y., McGrath, T., Powell, K., Pinglay, S., Burke, D. P., Goodarzi, H., Hsu, P. D., and Hie, B. L. Genome modelling and design across all domains of life with evo 2. *Nature*, 03 2026. ISSN 1476-4687. doi: 10.1038/s41586-026-10176-5. URL https://doi.org/10.1038/s41586-026-10176-5.

Bussmann, B., Leask, P., and Nanda, N. Batchtopk sparse autoencoders. In *NeurIPS 2024 Workshop on Scientific Methods for Understanding Deep Learning*, 2024. URL https://openreview.net/forum?id=d4dpOCqybL.

Bussmann, B., Nabeshima, N., Karvonen, A., and Nanda, N. Learning multi-level features with matryoshka sparse autoencoders. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=m25T5rAy43.

Chanin, D., Wilken-Smith, J., Dulka, T., Bhatnagar, H., Golechha, S., and Bloom, J. I. A is for absorption: Studying feature splitting and absorption in sparse autoencoders. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL https://openreview.net/forum?id=R73ybUciQF.

Costa, V., Fel, T., Lubana, E. S., Tolooshams, B., and Ba, D. E. Evaluating sparse autoencoders: From shallow design to matching pursuit. In *ICML 2025 Workshop on Methods and Opportunities at Small Scale*, 2025. URL https://openreview.net/forum?id=SLGftRJVUN.

Csordás, R., Potts, C., Manning, C. D., and Geiger, A. Recurrent neural networks learn to store and generate sequences using non-linear representations. In Belinkov, Y., Kim, N., Jumelet, J., Mohebbi, H., Mueller, A., and Chen, H. (eds.), *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pp. 248–262, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.blackboxnlp-1.17. URL https://aclanthology.org/2024.blackboxnlp-1.17/.

Cunningham, H., Ewart, A., Riggs, L., Huben, R., and Sharkey, L. Sparse autoencoders find highly interpretable features in language models. *ICLR*, 2024.

Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., et al. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022.

Farrell, T., Leask, P., and Moubayed, N. A. Order by scale: Relative-magnitude relational composition in attention-only transformers. In *Socially Responsible and Trustworthy Foundation Models at NeurIPS 2025*, 2025. URL https://openreview.net/forum?id=vWRVzNtk7W.

Feng, J. and Steinhardt, J. How do language models bind entities in context? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=zb3b6oKO77.

Ferrando, J., Sarti, G., Bisazza, A., and Costa-Jussà, M. R. A primer on the inner workings of transformer-based language models. *arXiv preprint arXiv:2405.00208*, 2024.

Gao, L., la Tour, T. D., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., and Wu, J. Scaling and evaluating sparse autoencoders. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=tcsZt9ZNKD.

Huben, R., Cunningham, H., Smith, L. R., Ewart, A., and Sharkey, L. Sparse autoencoders find highly interpretable features in language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=F76bwRSLeK.

Kanerva, P. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 1 (2):139–159, 2009. doi: 10.1007/s12559-009-9009-8.

Karvonen, A., Rager, C., Lin, J., Tigges, C., Bloom, J. I., Chanin, D., Lau, Y.-T., Farrell, E., McDougall, C. S., Ayonrinde, K., Till, D., Wearden, M., Conmy, A., Marks, S., and Nanda, N. SAEBench: A comprehensive benchmark for sparse autoencoders in language model interpretability. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=qrU3yNfX0d.

Leask, P., Bussmann, B., Pearce, M. T., Bloom, J. I., Tigges, C., Moubayed, N. A., Sharkey, L., and Nanda, N. Sparse autoencoders do not find canonical units of analysis. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL https://openreview.net/forum?id=9ca9eHNrdH.

Leask, P., Nanda, N., and Moubayed, N. A. Inference-time decomposition of activations (ITDA): A scalable approach to interpreting large language models. In *Forty-second International Conference on Machine Learning*, 2025b. URL https://openreview.net/forum?id=4WMZqAoYi4.

Lieberum, T., Rajamanoharan, S., Conmy, A., Smith, L., Sonnerat, N., Varma, V., Kramar, J., Dragan, A., Shah, R., and Nanda, N. Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2. In Belinkov, Y., Kim, N., Jumelet, J., Mohebbi, H., Mueller, A., and Chen, H. (eds.), *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pp. 278–300, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.blackboxnlp-1.19. URL https://aclanthology.org/2024.blackboxnlp-1.19/.

Marks, S., Karvonen, A., and Mueller, A. dictionary_learning, 2024. URL https://github.com/saprmarks/dictionary_learning.

Marks, S., Treutlein, J., Bricken, T., Lindsey, J., Marcus, J., Mishra-Sharma, S., Ziegler, D., Ameisen, E., Batson, J., Belonax, T., et al. Auditing language models for hidden objectives. *arXiv preprint arXiv:2503.10965*, 2025.

McDougall, C., Conmy, A., Kramár, J., Lieberum, T., Rajamanoharan, S., and Nanda, N. Gemma scope 2: Technical paper. Technical report, Google DeepMind, 9 2025. URL https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/gemma-scope-2-helping-the-ai-safety-community-deepen-understanding-of-complex-language-model-behavior/Gemma_Scope_2_Technical_Paper.pdf.

Movva, R., Peng, K., Garg, N., Kleinberg, J., and Pierson, E. Sparse autoencoders for hypothesis generation. In *Forty-*

second International Conference on Machine Learning, 2025. URL https://openreview.net/forum?id=4R0pugRyN5.

Nanda, N., Chan, L., Lieberum, T., Smith, J., and Steinhardt, J. Progress measures for grokking via mechanistic interpretability. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=9XFSbDPmdW.

Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., and Carter, S. Zoom in: An introduction to circuits. *Distill*, 2020. doi: 10.23915/distill.00024.001. https://distill.pub/2020/circuits/zoom-in.

Olsson, C., Elhage, N., Nanda, N., Joseph, N., DasSarma, N., Henighan, T., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Johnston, S., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html.

Paulo, G. S., Mallen, A. T., Juang, C., and Belrose, N. Automatically interpreting millions of features in large language models. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=EemtbhJOXc.

Plate, T. A. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3):623–641, 1995. doi: 10.1109/72.377968.

Prakash, N., Shaham, T. R., Haklay, T., Belinkov, Y., and Bau, D. Fine-tuning enhances existing mechanisms: A case study on entity tracking. In *Proceedings of the 2024 International Conference on Learning Representations*, 2024. arXiv:2402.14811.

Quirke, L., Shabalin, S., and Belrose, N. Binary sparse coding for interpretability. *arXiv preprint arXiv:2509.25596*, 2025.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Rajamanoharan, S., Conmy, A., Smith, L., Lieberum, T., Varma, V., Kramar, J., Shah, R., and Nanda, N. Improving sparse decomposition of language model activations with gated sparse autoencoders. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a. URL https://openreview.net/forum?id=zLBlin2zvW.

Rajamanoharan, S., Lieberum, T., Sonnerat, N., Conmy, A., Varma, V., Kramár, J., and Nanda, N. Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024b.

Smolensky, P. Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial Intelligence*, 46(1-2):159–216, 1990. doi: 10.1016/0004-3702(90)90007-M.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Wang, K. R., Variengien, A., Conmy, A., Shlegeris, B., and Steinhardt, J. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=NpsVSN6o4ul.

Wattenberg, M. and Viégas, F. Relational composition in neural networks: A survey and call to action. In *ICML 2024 Workshop on Mechanistic Interpretability*, 2024. URL https://openreview.net/forum?id=zzCEiUIPk9.

A. Further SAE Latent Analysis

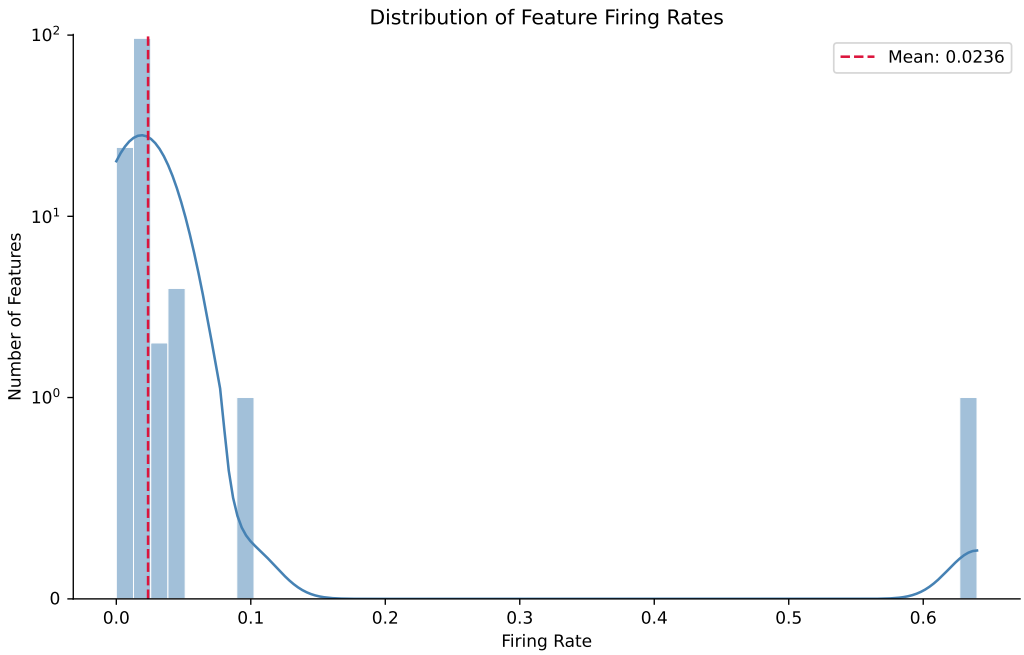

Figure 6. This histogram shows the distribution of firing rates for the SAE checkpoint's latents. The y-axis uses a symmetric logarithmic scale with a linear scale between $[-1, 1]$. A given latent's firing rate is the proportion of inputs which cause that latent to activate. The special latents are the only ones with a firing rate greater than one standard deviation from the mean (mean: 0.0236, standard deviation: 0.0559): latent 11 has rate 0.6400, latent 56 has rate 0.0991.

blabla

However, in JumpReLU test we find a latent that activates more than the specials but doesnt seem to be a special (no alpha_diff corr or scaling)

B. Further Latent Steering Examples

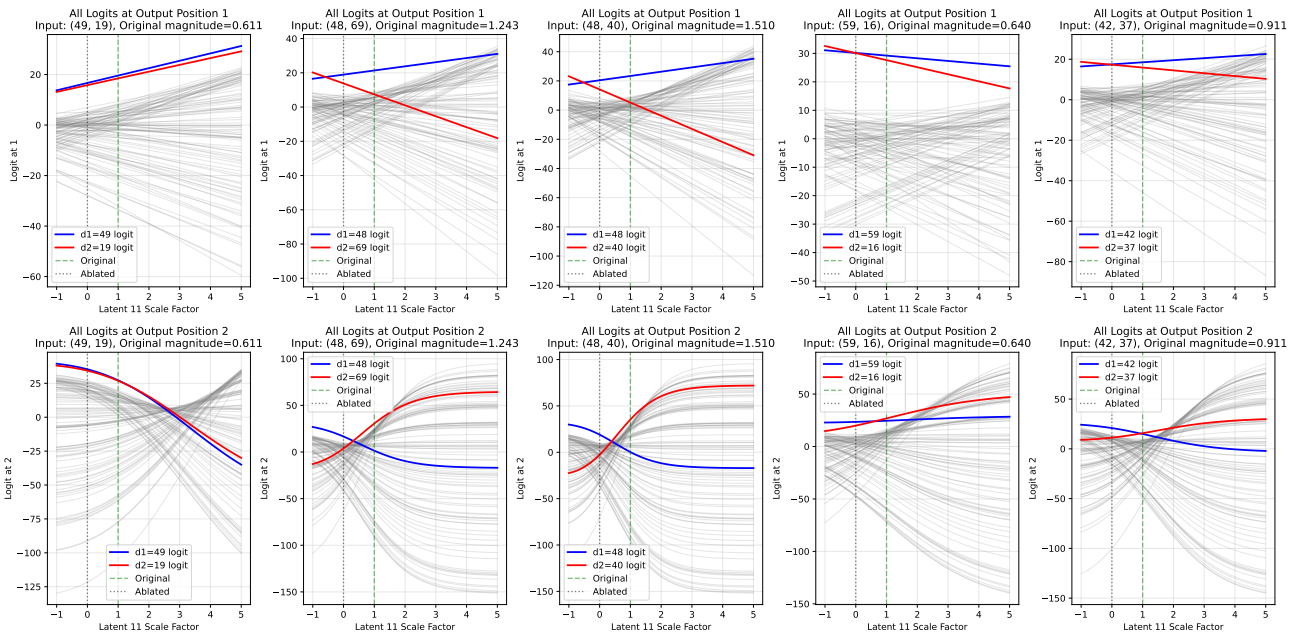

Figure 7. For 9.2% of inputs, latent 11 (in the BatchTopK SAE from Section **??**) must be negatively scaled to swap the predicted symbol at one of the output positions.

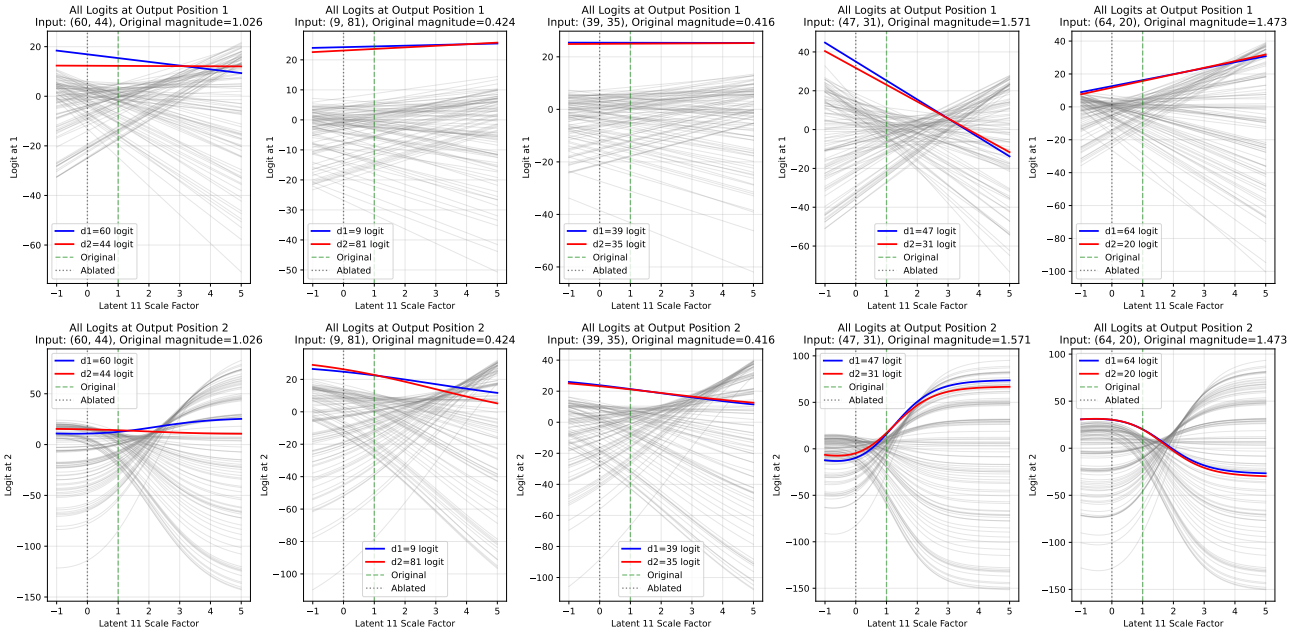

Figure 8. For 2.3% of inputs, it is possible to scale latent 11 (in the BatchTopK SAE from Section **??**) to swap the symbol at each output position individually but not simultaneously.

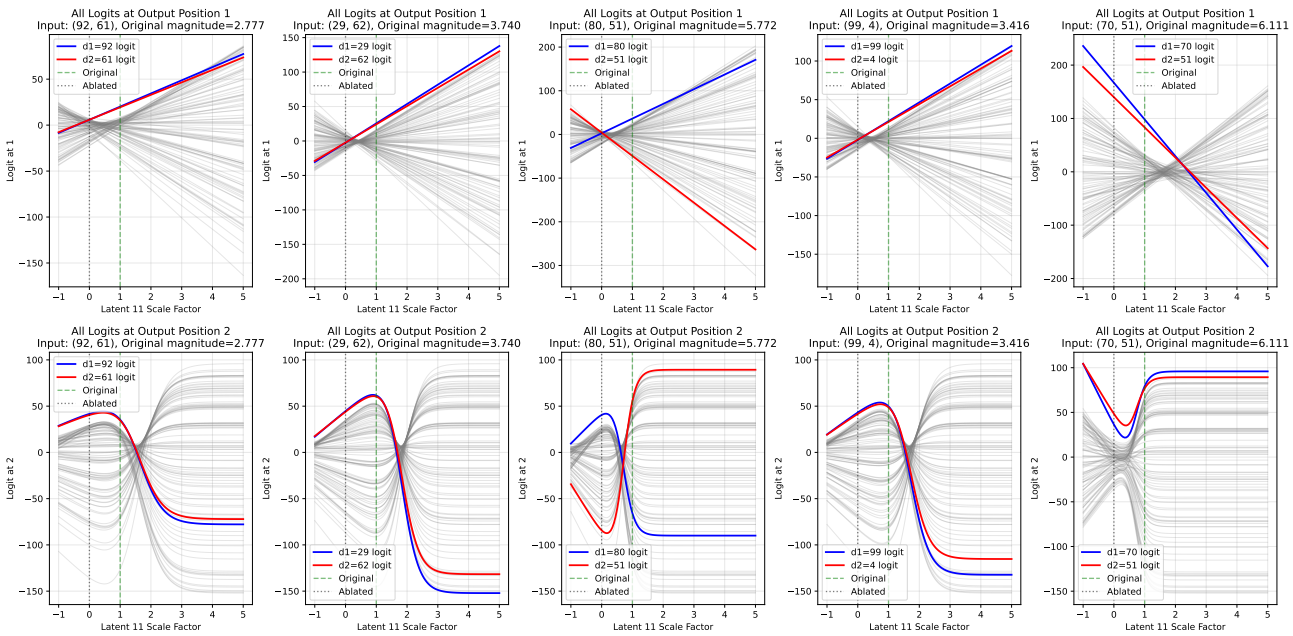

Figure 9. For 1.6% of inputs, the logit for the symbol at d_2 is always dominant at o_2 in the observed latent scale range (in the BatchTopK SAE from Section **??**), so there is no scale where the the symbol at d_1 is predicted instead.

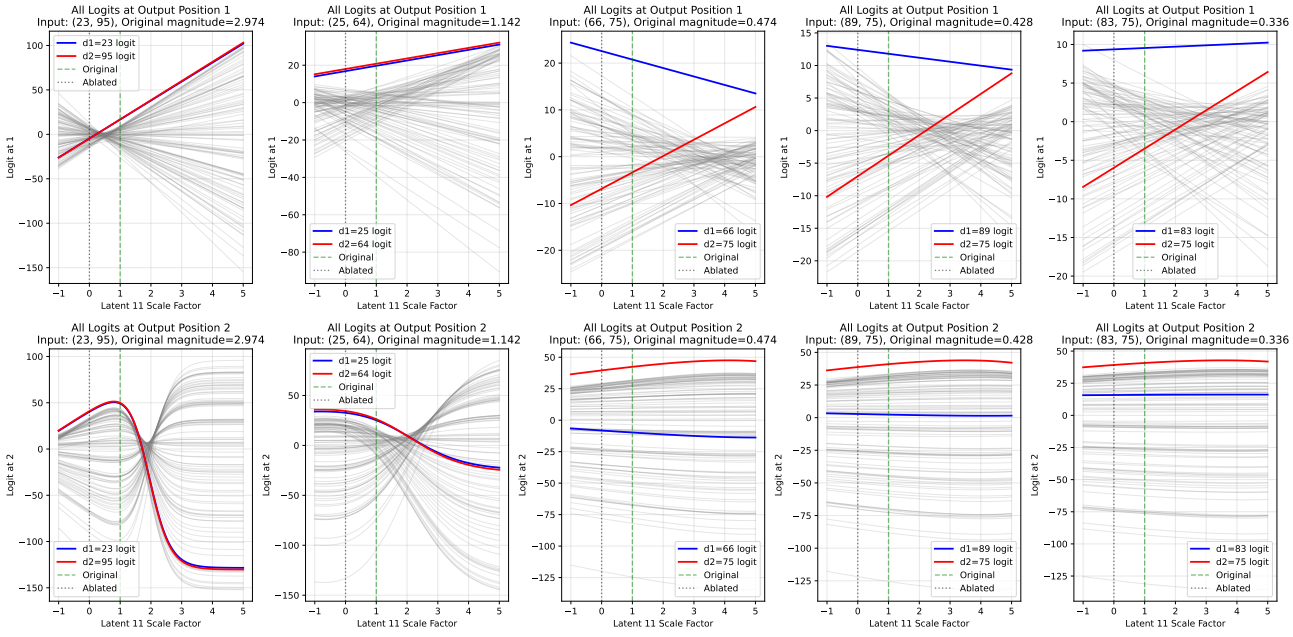

Figure 10. For 0.2% of inputs, the logit for the symbol at d_1 is always dominant at o_1 in the observed latent scale range (in the BatchTopK SAE from Section **??**), so there is no scale where the the symbol at d_2 is predicted instead.

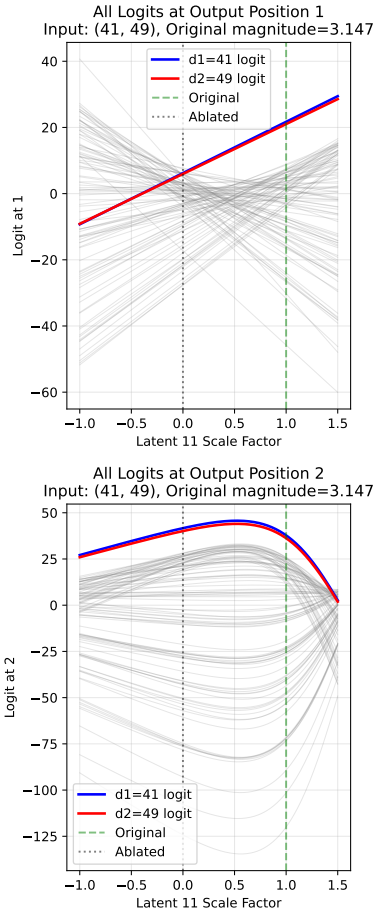

Figure 11. In the BatchTopK SAE from Section **??**, latent 11 cannot be scaled to swap input bigram $(41, 49)$ because the logit for 49 is never argmax in position o_1, as shown by the top plot.

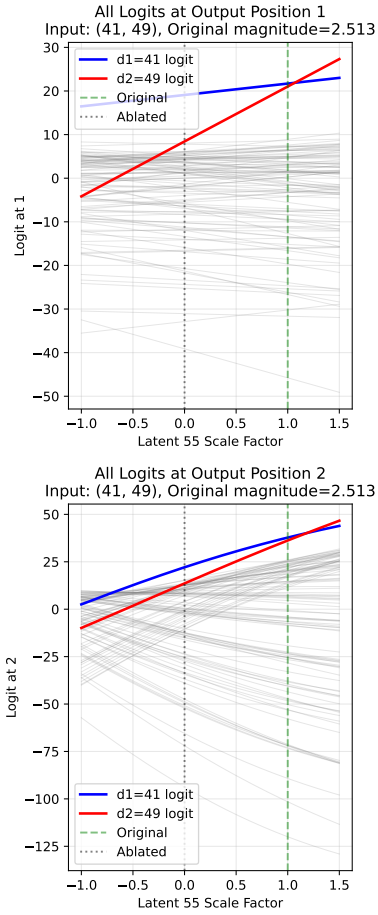

Figure 12. However, in the same SAE, latent 55 co-activates with latent 11 for input $(41, 49)$ and can be scaled by factor $p \in [1.07, 1.18]$ to cause the model to output $(49, 41)$ instead.

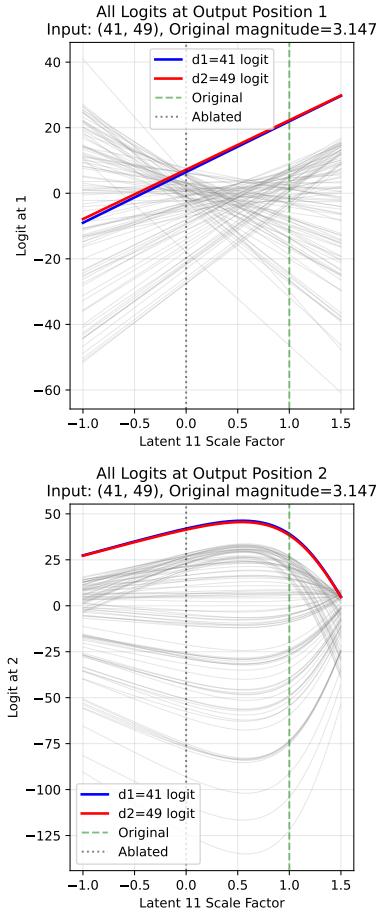

Figure 13. Moreover, in the same SAE, if latent 55's activation magnitude is scaled by a factor of 1.1, then we can now scale latent 11 by factor $p \in [0.03, 1.45]$ to swap the outputs, which was not possible without scaling latent 55. This suggests that some graded activation demonstrations are only possible when multiple active latents are scaled simultaneously.